

UNIT-4

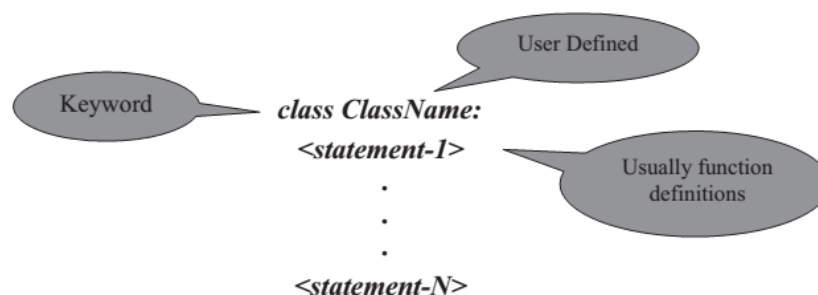
OOP

Classes and Objects in Programming

- In Python every type is a class. So int, float, complex, bool, str, list, tuple, set, dict are all classes.
- A class has a name, whereas objects are nameless. Since objects do not have names, they are referred using their addresses in memory.
- When we use a simple statement **num = 10**, a nameless object of type int is created in memory and its address is stored in num. Thus, num refers to or points to the nameless object containing value 10.
- However, instead of saying that num refers to a nameless int object, often for sake of convenience, it is said that num is an int object.
- The specific data in an object is often called **instance data** or **properties** of the object or **state** of the object or attributes of the object. Methods in an object are called **instance methods**.

Creating Classes in Python

The simplest form of class definition looks like this:



In practice, the statements inside a class definition will usually be **function definitions**, but few other statements are allowed. Because these functions are indented under a class, they are called **methods**. Methods are a special kind of function that is defined within a class.

Program to Illustrate Class and Object Creation

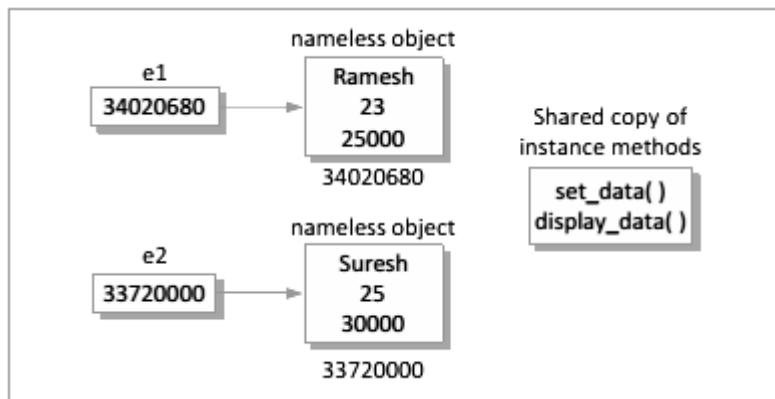
```
class Employee :  
    def set_data(self, n, a, s) :  
        self.name = n  
        self.age = a  
        self.salary = s  
    def display_data(self) :  
        print(self.name, self.age, self.salary)
```

```
e1 = Employee()  
e1.set_data('Ramesh', 23, 25000)  
e1.display_data()  
e2 = Employee()  
e2.set_data('Suresh', 25, 30000)  
e2.display_data()
```

Output

'Ramesh', 23, 25000

'Suresh', 25, 30000



- Whenever we call an instance method using an object, address of the object gets passed to the method implicitly. This address is collected by the instance method in a variable called **self**.
- **self** is like **this** pointer of C++ or **this** reference of Java. In place of self any other variable name can be used.

Access Convention

In Python, there are no keywords like **public**, **protected** or **private**. All attributes and methods are **public by default**.

There is one way to define private in Python. The syntax to define private attribute and methods is:

```
__Attribute  
__Methods_Name()
```

To make an attribute and a method private, we need to add two underscores, i.e. “__” in front of the attribute and the method’s name. It helps in hiding these when accessed out of class.

Program to illustrate the use of private

```
class Person:  
    def __init__(self):  
        self.Name = 'Bill Gates'           #Public attribute  
        self.__BankAccNo =10101           #Private attribute  
    def Display(self):  
        print(' Name = ',self.Name)  
        print('Bank Account Number = ',self.__BankAccNo)
```

```
P = Person()  
#Access public attribute outside class  
print(' Name = ',P.Name)  
P.Display()  
#Try to access private variable outside class but fails  
print(' Salary = ',P.__BankAccNo)  
P.Display()
```

Output

```
Name = Bill Gates
Name = Bill Gates
Bank Account Number = 10101
Traceback (most recent call last): #Error
  File "C:/Python34/PrivateDemo.py", line 13, in <module>
    print(' Salary = ',P.__BankAccNo)
AttributeError: 'Person' object has no attribute '__BankAccNo'
```

Note: Python hides a private name by changing its name to `__ClassName__AttrName`. This technique is termed as “**mangling**”.

Object Initialization

There are two ways to initialize an object:

Method 1 : Using methods like `get_data( )` / `set_data( )`.

Method 2 : Using special method `__init__( )`

`get_data( )` can receive data from keyboard into instance data variables.

`set_data( )` can set up instance data with a values that it receives.

The benefit of this method is that the data remains protected from manipulation from outside the class.

The benefit of initializing an object using the special method `__init__( )` is that it guarantees initialization, since `__init__( )` is always called when an object is created.

Following program illustrates both these methods:

```
class Employee :
    def set_data(self, n, a, s) :
        self.__name = n
        self.__age = a
        self.__salary = s
    def display_data(self) :
        print(self.__name, self.__age, self.__salary)
    def __init__(self, n = ' ', a = 0, s = 0.0) :
        self.__name = n
        self.__age = a
        self.__salary = s
    def __del__(self) :
        print('Deleting object' + str(self))
```

```
e1 = Employee( )
e1.set_data('Suresh', 25, 30000)
e1.display_data( )
e2 = Employee('Ramesh', 23, 25000)
e2.display_data( )
e1 = None
e2 = None
```

Output

Ramesh 23 25000

Suresh 25 30000

Deleting object<__main__.Employee object at 0x013F6810>

Deleting object<__main__.Employee object at 0x013F65B0>

- Whenever an object is created, space is allocated for it in memory and `__init__( )` is called. So address of object is passed to `__init__( )`.
- `__init__( )`'s parameters can take default values. In our program they get used while creating object e2.
- `__init__( )` doesn't return any value.
- If we do not define `__init__( )`, then Python inserts a default `__init__( )` method in our class.
- `__init__( )` is called only once during entire lifetime of an object.
- A class may have `__init__( )` as well as `set_data( )`.
`__init__( )` – To initialize object.
`set_data( )` – To modify an already initialized object.
- `__del__( )` method gets called automatically when an object goes out of scope. Cleanup activity, if any, should be done in `__del__( )`.
- `__init__( )` method is similar to **constructor** function of C++.
- `__del__( )` is similar to **destructor** function of C++.

Class Variables and Methods

- If we wish to share a variable amongst all objects of a class, we must declare the variable as a class variable or class attribute.
- To declare a class variable, we have to create a variable without prepending it with self.
- Class variables do not become part of objects of a class.
- Class variables are accessed using the syntax `classname.varname`.
- Contrasted with instance methods, class methods do not receive a self argument.
- Class methods can be accessed using the syntax: **`classname.methodname( )`**.
- Class variables can be used to count how many objects have been created from a class.
- Class variables and methods are like static members in C++ / Java.

Method Overloading in Python

Most object-oriented programming languages contain the concept of method overloading. It simply refers to having multiple methods with the same name which accept different sets of arguments.

C++ and Java support method overloading. Both the languages allow more than one methods with same name and different signature. The type of method argument defines the signature. In case of overloading, the signature determines which method is actually being invoked.

However, Python does not allow method overloading based on type as it is not strongly typed language.

In Python, method overloading can be achieved by using the inbuilt function **`isinstanceof`**.

Program on method overloading

```
class Demo:
    result = 0
    def add(self,instanceOf=None, *args):
        if instanceOf == 'int':
            self.result = 0
        if instanceOf == 'str':
            self.result = ""
        for i in args:
            self.result = self.result + i
        return self.result
```

```
D1=Demo()
print(D1.add('int', 10,20,30))
print(D1.add('str', 'I ', ' Love ', ' Python ', ' Programming '))
```

Output

```
60
I Love Python Programming
```

- The instanceof method checks the type of the first parameter being passed to the add() method. It stores the value of result process based upon **type**.